

Trabalho Conceitual — YAML e a Orquestração de Containers com Kubernetes

Isa

Julho de 2026

1. O que é YAML

YAML (acrônimo recursivo de *YAML Ain't Markup Language*) é um formato de serialização de dados projetado para ser legível por seres humanos. Diferente de formatos como XML, que dependem de tags de abertura e fechamento, ou JSON, que usa chaves e colchetes, o YAML organiza a informação por meio de **indentação** (espaços no início das linhas), o que torna a leitura do arquivo muito próxima da leitura de um texto estruturado em tópicos. Por essa característica, ele se tornou o formato padrão para arquivos de configuração em diversas ferramentas modernas de infraestrutura, entre elas o Kubernetes.

2. Estrutura do YAML

A estrutura do YAML se apoia em três elementos básicos. O primeiro são os **pares chave-valor** (replicas: 2), que associam um nome a um dado. O segundo são as **listas**, indicadas por um hífen no início da linha (- containerPort: 80), usadas quando um campo aceita vários itens. O terceiro é o **aninhamento por indentação**: um campo deslocado com espaços à direita “pertence” ao campo acima dele, formando uma hierarquia. Uma regra importante é que a indentação deve ser feita sempre com espaços, nunca com *tab*, e de forma consistente — um único espaço fora do lugar altera o significado do arquivo.

3. Vantagens e importância na orquestração com Kubernetes

No Kubernetes, os arquivos YAML são chamados de **manifestos** e seguem uma anatomia comum de quatro campos principais: `apiVersion` (a versão da API do Kubernetes usada para interpretar o objeto), `kind` (o tipo de objeto, como *Deployment* ou *Service*), `metadata` (nome e rótulos que identificam o objeto) e `spec` (a especificação do estado desejado). O ponto central é que o manifesto é **declarativo**: em vez de listar comandos a executar, ele descreve *como o ambiente deve ficar*, e o próprio Kubernetes trabalha continuamente para manter esse estado — se um container falha, outro é criado automaticamente para restabelecer o que foi declarado.

As principais vantagens dessa abordagem são a **legibilidade** (a equipe inteira entende a infraestrutura lendo o arquivo), a **replicabilidade** (o mesmo manifesto sobe o mesmo ambiente em qualquer cluster), o **versionamento** (os arquivos podem ser guardados no Git, praticando infraestrutura como código, com

histórico e revisão de mudanças) e a **escalabilidade** (alterar replicas: 2 para replicas: 4 e reaplicar o arquivo é suficiente para dobrar a capacidade de um serviço). Empresas como Spotify e iFood migraram suas plataformas de máquinas virtuais para Kubernetes justamente para obter essa elasticidade e padronização em larga escala — o mesmo movimento proposto neste desafio para a Web Solutions Ltda.

4. Passo a passo da execução da automação

A solução implementada colocou dois servidores web independentes — **Nginx (porta 8080)** e **Apache HTTPD (porta 8081)** — em um único cluster Kubernetes local (Minikube, com driver Docker), da seguinte forma:

1. **Criação das imagens Docker:** dois Dockerfile baseados em imagens oficiais otimizadas (nginx:alpine e httpd:alpine, variantes leves), cada um copiando uma página HTML personalizada para o diretório público do servidor. Build com `docker build -t webso-nginx:1.0 nginx/` e `docker build -t webso-apache:1.0 apache/` (com `eval $(minikube docker-env)` antes, para o cluster enxergar as imagens locais).
2. **Manifestos Kubernetes:** quatro arquivos YAML — um *Deployment* e um *Service* para cada servidor. Os Deployments declaram 2 réplicas de cada aplicação; os Services, do tipo NodePort, expõem o Nginx na porta 8080 (nodePort 30080) e o Apache na 8081 (nodePort 30081), direcionando o tráfego pelos rótulos (labels/selector).
3. **Implantação:** `kubectl apply -f k8s/` aplica os quatro manifestos de uma vez; `kubectl get pods` e `kubectl get services` confirmam os pods em estado *Running* e as portas expostas.
4. **Acesso e teste:** com `kubectl port-forward service/nginx-service 8080:8080` e `kubectl port-forward service/apache-service 8081:8081`, as aplicações ficam acessíveis no navegador em `localhost:8080` e `localhost:8081`, cada uma exibindo sua página, comprovando a coexistência dos dois serviços de forma independente no mesmo cluster.
5. **Escalabilidade:** o comando `kubectl scale deployment nginx-deployment --replicas=4` demonstra o escalonamento independente de um serviço sem afetar o outro.

5. Conclusão

O YAML é a “linguagem de contrato” entre a equipe e o orquestrador: nele declara-se o estado desejado da infraestrutura, e o Kubernetes o materializa e o mantém. Para a Web Solutions, isso significa trocar implantações manuais, lentas e sujeitas a conflitos por um processo declarativo, versionável, replicável e escalável — e a mesma estrutura de manifestos usada nesta prova de conceito pode ser expandida para migrar as demais aplicações da empresa.

Referências

Documentação oficial do Docker (docs.docker.com); documentação oficial do Kubernetes — Concepts: Pods, Deployments, Services (kubernetes.io/docs); especificação YAML (yaml.org); case de mercado: Spotify — migração para Kubernetes (kubernetes.io/case-studies/spotify).